

Cloud Computing Architecture

Semester project report

Group 24

Orofino Antonino - 22-956-908

Paleari Lorenzo - 22-943-047

Sainz Martinez Julián - 22-953-467

Systems Group
Department of Computer Science
ETH Zurich
April 5, 2023

Instructions

- Please do not modify the template, except for putting your solutions, group number, names and legi-NR.
- Parts 1 and 2 should be answered in maximum six pages (including the questions).
If you exceed the space, points may be subtracted.

Part 1 [25 points]

Using the instructions provided in the project description, run memcached alone (i.e., no interference), and with each iBench source of interference (cpu, l1d, l1i, l2, llc, membw). For Part 1, you must use the following `mcperf` command, which varies the target QPS from 30000 to 110000 in increments of 5000 (and has a warm-up time of 2 seconds with the addition of `-w 2`):

```
$ ./mcperf -s MEMCACHED_IP --loadonly
$ ./mcperf -s MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 16 -C 4 -D 4 -Q 1000 -c 4 -w 2 -t 5 \
    --scan 30000:110000:5000
```

Repeat the run for each of the 7 configurations (without interference, and the 6 interference types) **at least three times** (three should be sufficient in this case), and collect the performance measurements (i.e., the `client-measure` VM output). Reminder: after you have collected all the measurements, make sure you delete your cluster. Otherwise, you will easily use up the cloud credits. See the project description for instructions how.

(a) [10 points] Plot a single line graph with the following stipulations:

- Queries per second (QPS) on the x-axis (the x-axis should range from 0 to 110K). (note: the actual achieved QPS, not the target QPS)
- 95th percentile latency on the y-axis (the y-axis should range from 0 to 8 ms).
- Label your axes.
- 7 lines, one for each configuration. Add a legend.
- State how many runs you averaged across and include error bars at each point in both dimensions.
- The readability of your plot will be part of your grade.

Solution: In order to plot the graph we averaged three different runs: as we only have three measurements the error bars represent the minimum and maximum measurements taken, while the point represents the average. We could represent the error bars as the standard deviation but with our approach we maintain all the information in the measurements as they are represented directly in the graph. For the sake of readability, we averaged several measurements with achieved QPS very similar one to another in the `l1i` and `cpu` benchmarks as they struggle to reach more than 50K we would have an agglomerate of points there. In this way, we do not have too many points concentrated in the same area of the graph thus improving readability. This is also the reason why the error bars are bigger in this part of the graph as we are averaging over several results.

Memcached 95th Percentile Latency

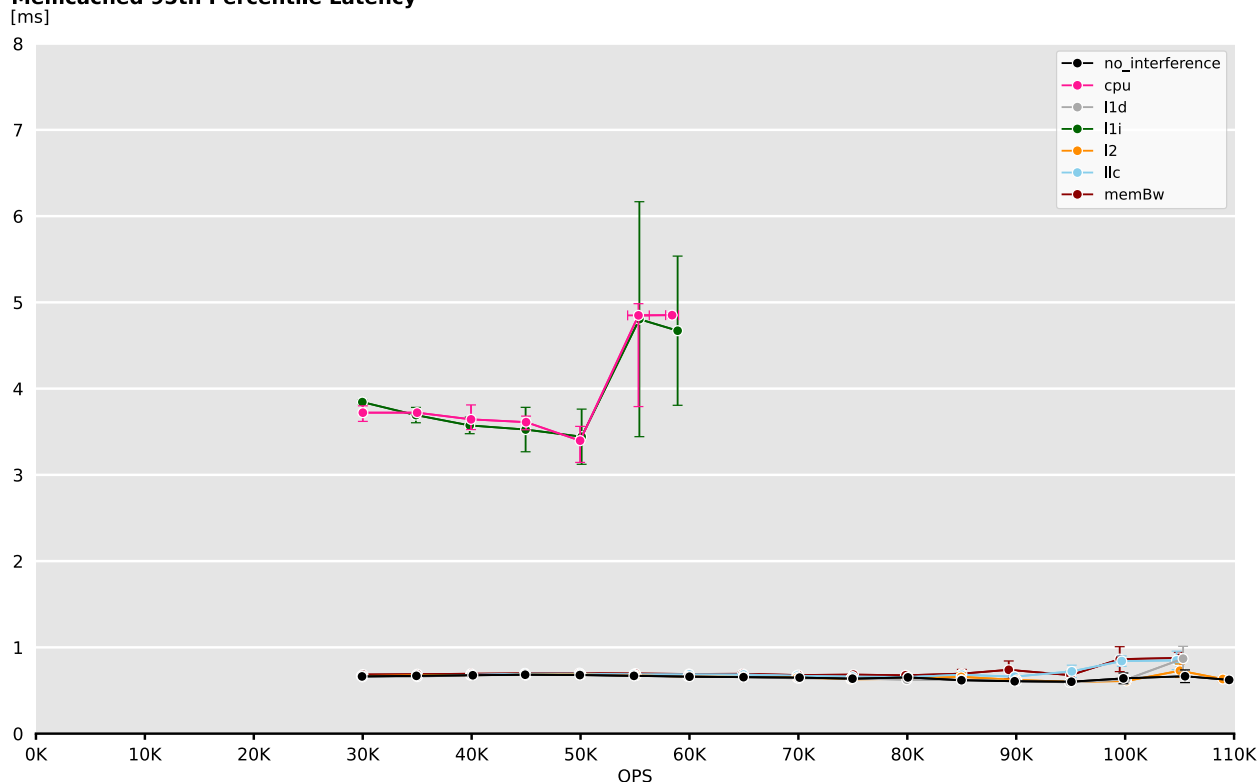


Figure 1: 95th Percentile Latency

- (b) [6 points] How is the tail latency and saturation point (the “knee in the curve”) of memcached affected by each type of interference? Why? Briefly describe your hypothesis.

Solution: **memcached** reaches saturation point around 55K with **cpu** and **l1i** benchmarks. From time to time it manages to get to 60K QPS, but this is anecdotal. Therefore, we can conclude that **memcached** makes a huge use of CPU operation and has to process lots of instructions.

The application seems to be less memory dependent as **l1d**, **l2**, **l1c** and **memBw** have a much smaller latency even if **textttl1c** and **memBw** start to get a slightly bigger latency at 90K. This is at first glance unexpected as **memcached** is by definition using the memory to cache database values and has to retrieve them frequently. In fact **memBw** and **l1c** have the worst performance among this group of benchmarks, which point to the mentioned memory dependency. **l1d**, **l1c** and **memBw** seem to reach the saturation point at 105K which may indicate that higher QPS could lead to a big drop in performance. All in all, our hypothesis for this better-than-expected performance is that the memory bandwidth in the VM is big enough to handle the transfer of the small chunks of data that **memcached** retrieves from memory.

- (c) [2 points] Explain the use of the **taskset** command in the container commands for memcached and iBench in the provided scripts. Why do we run some of the iBench benchmarks on the same core as memcached and others on a different core?

Solution: **taskset** allows to set the core on which each benchmark will be run, more precisely it uses a mask to specify the processor affinity of a process, that is the set of CPUs a thread is eligible to run on. Thus we can divide the benchmark in two groups, the one that have to run

on the same core as memcached and the one that aren't supposed to. It is straightforward to conclude that we should run the **cpu** benchmark on the same core of **memcached**, as the task of this benchmark is to strain the CPU. Furthermore, **11i**, **11d** and **12** are not shared among cores so they also have to be run in the same core as the one used by the application that wants to be tested. Nevertheless, they are not really CPU intensive so they don't create additional interference. On the other hand, **11c** and **memBW** can and should be run on a separate core as these resources are shared among cores and these benchmarks are more CPU intensive and this would create additional interference.

- (d) [2 points] Assuming a service level objective (SLO) for memcached of up to 1.5 ms 95th percentile latency at 65K QPS, which iBench source of interference can safely be collocated with memcached without violating this SLO? Briefly explain your reasoning.

Solution: Taking a look at the graph we can see that at 65K QPS **11i** and **cpu** cannot be collocated with memcached without violating the SLO of up to 1.5 ms 95th percentile given that they are well above that time limit, around 5 ms. They even struggle to process more than 50K QPS. **11d**, **12**, **11c** and **memBW** can be safely collocated as they are below 1 ms of tail latency, and do not struggle processing any number of QPS.

- (e) [5 points] In the lectures you have seen queueing theory. Is the project experiment above an open system or a closed system? What is the number of clients in the system? Sketch a diagram of the queueing system and provide an expression for the average response time. Explain each term in the response time expression.

Solution: This is clearly a closed system as the number of clients is well established. Based on the manual of memcache-perf we can see that we are running a total of 16 thread (-T 16) and each one of these threads open a total of 4 connections (-C 4) from the VM client-agent to the **memcached** server. One additional connection is opened in the client-measure VM that is in charge of taking the measurements. Therefore, the **total** number of clients is **65**. It is also stated in the manual that mcperv "waits for a reply before sending the next request". All in all, this makes it a closed system by definition.

In Figure 3 the queueing system of the described closed system is represented.

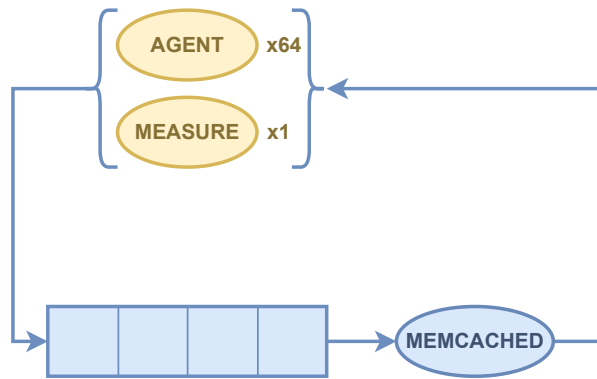


Figure 2: Queueing system

The expression for the average response time (R) is included below:

$$R = \frac{N}{X} - Z$$

N represents the number of clients in the closed system and Z represents the think time of the clients. In other words Z represents the possible delay between a client receiving an answer from the server and the time it sends the next request. In our case, if a think time is present in the clients, it would have to be really small as we limit the total runtime of sending and receiving requests to 5 seconds ($-t \leq 5$). Finally, $X = \frac{Jobs}{Time}$ is the throughput or number of queries per second.

Part 2 [30 points]

1. Interference behavior [19 points]

- (a) [11 points] Fill in the following table with the normalized execution time of each batch job with each source of interference. The execution time should be normalized to the job's execution time with no interference. Round the normalized execution time to 2 decimal places. Color-code each field in the table as follows: **green** if the normalized execution time is less than or equal to 1.3, **orange** if the normalized execution time is over 1.3 and up to 2, and **red** if the normalized execution time is greater than 2. Briefly summarize in a paragraph the resource interference sensitivity of each batch job.

Workload	none	cpu	l1d	l1i	l2	l1c	memBW
blackscholes	1.00	1.31	1.12	1.54	1.15	1.49	1.32
canneal	1.00	1.18	1.21	1.35	1.18	1.98	1.31
dedup	1.00	1.66	1.19	2.04	1.19	2.19	1.65
ferret	1.00	1.83	1.00	2.24	1.01	2.58	1.93
freqmine	1.00	2.01	1.02	2.00	1.04	1.90	1.61
radix	1.00	1.06	1.07	1.15	1.05	1.66	1.10
vips	1.00	1.73	1.60	1.98	1.60	2.09	1.76

Solution: As can be seen in the table all jobs are affected by the l1c interference. On the other hand, l1d and l2 are the tasks that interfere the least with the jobs. memBW also affect all of the jobs, but not in such a dramatic way as l1c. l1i has also a high influence on execution time, more than memBW, although it is better than l1c again, specially considering that radix is not very much affected by it. cpu can be considered slightly better than l1i.

considering a more detail, per application, look we can notice that **vips** is the most resource demanding task as it is affected by all sources of interference. The least resource demanding tasks are **radix** and **canneal** this could imply that they are neither very CPU or data dependent. Nevertheless, some critical data should always be accessed and, as a consequence, the penalty of accessing memory after interference to l1c is still noticeable. There are some similarities between **blackscholes**, **dedup**, **ferret**, and **freqmine** as they are all visibly CPU and data dependent with noticeable drops in performance, although the least affected one is **blackscholes**.

- (b) [8 points] Explain what the interference profile table tells you about the resource requirements for each application. Which jobs (if any) seem like good candidates to collocate with memcached from Part 1, without violating the SLO of 2 ms P95 latency at 40K QPS?

Solution: Given the poor performance of the applications when facing interference in l1c we can say that all of the tasks are data intensive. An overused l1c would imply a

cache miss and memory access which is very detrimental to execution time, much more than misses to the previous levels of cache. This last points is reflected in the table, given that **l1d** and **l2** interference do not make a big difference on execution time.

Additionally, we can see a clear correlation between **cpu** and **l1i**. Applications that are most affected by **cpu** interference are also the most affected by **l1i** interference. This clearly indicates that this tasks are in general computational intensive as they require to load a large number of instructions from cache and are CPU intensive. Following this thread, **radix** and **canneal** are clearly not very much CPU or memory dependent as they are not strained by their interference.

According to the graph: **ferret**, **dedup**, **freqmine** and **blacksholes** are in order the applications that are the most CPU and memory dependent.

Regarding the second part of the question, **memcached** was shown to be affected most by **cpu** and **l1i** interferences. Consequently, in order to not violate the SLO of 2 ms P95 latency at 40K QPS it would be best to colocate it with tasks that are not as dependent on CPU, given that at 40K the latency is already affected by these interferences. Thus, the most suitable application would be **radix**, followed closely by **canneal** because even if it is slightly affected by **l1i** we can still see that it is int the lower range of the orange color-code compared to the other tasks. Last, if needed, it could also be possible to colocate it with **blacksholes** for the same reasons as **canneal**, but it should be the last option among theese three since it is slightly worse than **canneal**.

2. Parallel behavior [11 points]

Plot a single line graph with speedup as the y-axis (normalized time to the single thread config, $\text{Time}_1 / \text{Time}_n$) vs. number of threads on the x-axis (1, 2, 4 and 8 threads, see the project description for more details). Briefly discuss the scalability of each application: e.g., linear/sub-linear/super-linear. Do the applications gain significant speedup with the increased number of threads? Explain what you consider to be “significant”.

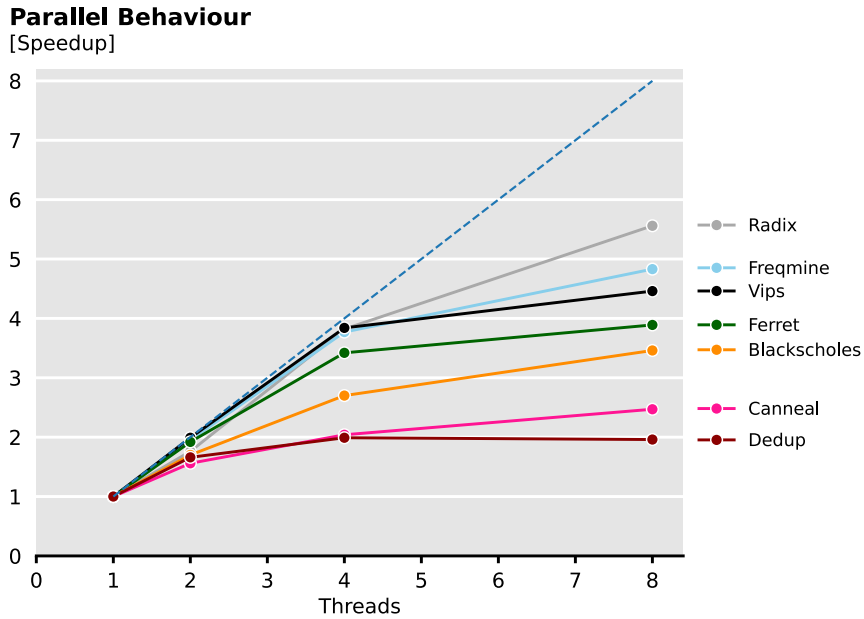


Figure 3: Parallelization trends

Solution: While all of the speedups are sublinear we can still notice some interesting facts. **canneal** and **dedup** are the tasks that benefit the least of the parallelization. While their speedup is significantly good with 4 threads as we obtained a 2 fold improvement, it is clearly not worth to add more threads as the performance stays the same or even gets slightly worse. This is probably due to the overhead of running and creating new threads. On the other hand, tasks **radix**, **freqmine** and **vips** are the most benefited in that order. Their parallelization offers diminishing returns as is expected by Amdahl's law, but it is still worth it to run 8 threads. Up to 4 threads the trend is almost linear, the slight difference is again probably due to the overhead of creating and running new threads. This graphical analysis can, therefore, help us figure out which applications are highly parallelizable and which are not allowing us to better distribute resources.